Python FASTAPI 폴더 및 파일 구조

처음에는 다음 다섯 가지만 이해하면 됩니다.

```
요청 받기
→ 처리하기
→ 데이터베이스(supabase)에 저장하기
→ 결과 돌려주기
```

이를 파일로 나누면 다음과 같습니다.

```
backend/
└── app/
    ├── main.py
    ├── database.py
    ├── models/
    ├── schemas/
    ├── services/
    └── routers/
```

1. 전체 구조

```
backend/
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── database.py
│   │
│   ├── models/
│   │   ├── __init__.py
│   │   └── user.py
│   │
│   ├── schemas/
│   │   ├── __init__.py
│   │   └── user.py
│   │
│   ├── services/
│   │   ├── __init__.py
│   │   └── user_service.py
│   │
│   └── routers/
│       ├── __init__.py
│       └── user_router.py
│
├── .env
├── requirements.txt
└── README.md
```

처음에는 이 정도 구조면 충분합니다.

---

2. 각 파일은 무슨 역할인가

`main.py`

서버를 시작하는 파일입니다.

```python
from fastapi import FastAPI

app = FastAPI()
```

FastAPI 애플리케이션을 생성하는 가장 중심이 되는 파일입니다.

서버 실행 명령어는 다음과 같습니다.

```
uvicorn app.main:app --reload
```

---

routers/

어떤 주소로 요청을 받을지 정하는 곳입니다.

예를 들어:

```
POST /users
GET /users/1
```

같은 API 주소를 작성합니다.

```python
# app/routers/user_router.py

from fastapi import APIRouter

router = APIRouter(prefix="/users")


@router.get("/{user_id}")
def get_user(user_id: int):
    return {
        "user_id": user_id
    }
```

이 파일은 사용자의 요청을 가장 먼저 받습니다.

다만 실제 작업을 전부 여기서 처리하지는 않습니다.

---

services/

실제 기능을 처리하는 곳입니다.

예를 들어 회원가입이라면 다음 작업이 필요합니다.

```
이메일 중복 확인
비밀번호 암호화
사용자 생성
사용자 저장
```

이런 기능을 Service에 작성합니다.

```python
# app/services/user_service.py

class UserService:

    def create_user(self, email: str, name: str):
        # 이메일 중복 검사
        # 사용자 생성
        # 데이터베이스 저장
        pass
```

쉽게 말하면 Router는 요청을 받고, Service는 실제 일을 처리합니다.

```
Router
"회원가입 요청이 들어왔다."

Service
"이메일을 확인하고 사용자를 생성하자."
```

---

models/

데이터베이스 테이블의 구조를 정의하는 곳입니다.

예를 들어 `users` 테이블이 다음과 같다고 하겠습니다.

컬럼	의미
`id`	사용자 번호
`email`	이메일
`name`	이름
`password_hash`	암호화된 비밀번호

이를 Python 클래스로 표현한 것이 Model입니다.

```python
# app/models/user.py

from sqlalchemy import String
from sqlalchemy.orm import Mapped, mapped_column

from app.database import Base


class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(
        primary_key=True
    )

    email: Mapped[str] = mapped_column(
        String(255),
        unique=True,
        nullable=False,
    )

    name: Mapped[str] = mapped_column(
        String(100),
        nullable=False,
    )

    password_hash: Mapped[str] = mapped_column(
        String(255),
        nullable=False,
    )
```

이 클래스는 데이터베이스의 `users` 테이블과 연결됩니다.

```
User 클래스
↕
users 테이블
```

`schemas/`

사용자가 보내는 데이터와 서버가 돌려주는 데이터의 형태를 정의합니다.

예를 들어 회원가입 요청은 다음과 같습니다.

```json
{
  "email": "junho@example.com",
  "name": "준호",
  "password": "password123"
}
```

이 요청 형식을 Schema로 작성합니다.

```python
# app/schemas/user.py

from pydantic import BaseModel, EmailStr

class UserCreateRequest(BaseModel):
    email: EmailStr
    name: str
    password: str
```

사용자 정보를 응답할 때는 비밀번호를 포함하면 안 됩니다.

```python
class UserResponse(BaseModel):
    id: int
    email: EmailStr
    name: str
```

따라서 Schema는 보통 요청용과 응답용을 나눕니다.

```
UserCreateRequest
→ 사용자가 서버에 보내는 데이터

UserResponse
→ 서버가 사용자에게 보내는 데이터
```

---

database.py

FastAPI 서버와 데이터베이스를 연결하는 파일입니다.

```python
# app/database.py

from sqlalchemy.ext.asyncio import (
    create_async_engine,
    async_sessionmaker,
    AsyncSession,
)
from sqlalchemy.orm import DeclarativeBase


DATABASE_URL = (
    "postgresql+asyncpg://"
    "postgres:password@localhost:5432/my_app"
)


class Base(DeclarativeBase):
    pass


engine = create_async_engine(DATABASE_URL)

SessionFactory = async_sessionmaker(
    engine,
    class_=AsyncSession,
    expire_on_commit=False,
)


async def get_db():
```

```
    async with SessionFactory() as session:
        yield session
```

처음에는 세부 내용을 전부 이해할 필요는 없습니다.

핵심은 다음입니다.

```
DATABASE_URL
→ 데이터베이스 주소

engine
→ 데이터베이스 연결 담당

session
→ 실제 조회와 저장에 사용하는 객체
```

---

3. User 기능은 왜 여러 파일로 나누는가

User와 관련된 파일은 다음처럼 나뉩니다.

```
models/user.py
schemas/user.py
services/user_service.py
routers/user_router.py
```

처음 보면 같은 User 기능을 왜 여러 파일에 나누는지 의문이 들 수 있습니다.

각각 역할이 다르기 때문입니다.

```
models/user.py
→ users 테이블 구조

schemas/user.py
→ 요청과 응답 데이터 구조

services/user_service.py
→ 사용자 관련 기능

routers/user_router.py
→ 사용자 관련 API 주소
```

예를 들어 회원가입 기능을 만들면 다음과 같이 흐릅니다.

```
Flutter가 회원가입 요청 전송
        ↓
user_router.py가 요청 받음
        ↓
user_service.py가 회원가입 처리
        ↓
user.py Model을 이용해 DB 저장
        ↓
UserResponse 형태로 결과 반환
```

---

4. 실제 회원가입 요청 흐름

Flutter가 다음 데이터를 보냅니다.

```
{
  "email": "junho@example.com",
  "name": "준호",
  "password": "password123"
}
```

1단계: Router가 요청을 받음

```python
# app/routers/user_router.py

from fastapi import APIRouter

from app.schemas.user import (
    UserCreateRequest,
    UserResponse,
)

router = APIRouter(prefix="/users")

@router.post("", response_model=UserResponse)
async def create_user(
    request: UserCreateRequest,
):
    pass
```

여기서 `UserCreateRequest`가 요청 데이터를 검사합니다.

이메일 형식이 잘못되었거나 필요한 값이 없으면 FastAPI가 자동으로 오류를 반환합니다.

2단계: Service가 요청 처리

```python
# app/services/user_service.py

from app.schemas.user import UserCreateRequest

class UserService:
    async def create_user(
        self,
        request: UserCreateRequest,
    ):
        # 사용자 생성 처리
        pass
```

Service에서는 다음과 같은 작업을 수행합니다.

```
이메일 중복인지 확인
비밀번호 암호화
User 객체 생성
데이터베이스 저장
```

3단계: 결과 반환

최종적으로 서버는 다음과 같은 응답을 보냅니다.

```json
{
  "id": 1,
  "email": "junho@example.com",
  "name": "준호"
}
```

비밀번호는 응답하지 않습니다.

---

5. `__init__.py` 는 무엇인가

각 폴더 안에는 보통 다음 파일이 있습니다.

```
__init__.py
```

예를 들면:

```
models/
├── __init__.py
└── user.py
```

이 파일은 Python에게 해당 폴더가 Python 파일을 묶어둔 패키지라는 것을 알려줍니다.

내용은 비어 있어도 됩니다.

```
# 빈 파일이어도 됨
```

이 파일이 있으면 다음처럼 다른 파일을 가져올 수 있습니다.

```
from app.models.user import User
```

---

6. 기능이 추가되면 어떻게 되는가

게시글 기능을 추가한다고 하겠습니다.

그러면 기존 구조에 파일을 하나씩 추가합니다.

```
models/
├── user.py
└── post.py

schemas/
├── user.py
└── post.py

services/
├── user_service.py
└── post_service.py

routers/
├── user_router.py
└── post_router.py
```

즉 기능 하나마다 관련 파일이 추가됩니다.

```
User 기능
→ user.py

Post 기능
→ post.py

Notification 기능
→ notification.py
```

---

7. 처음부터 Repository까지 만들어야 하는가

프로젝트 구조에 `repositories/` 폴더를 추가하기도 합니다.

```
repositories/
└── user_repository.py
```

Repository는 데이터베이스 조회와 저장만 담당합니다.

하지만 백엔드를 처음 배우는 단계에서는 반드시 필요한 것은 아닙니다.

처음에는 다음 구조로 시작해도 됩니다.

```
Router
→ Service
→ Database
```

프로젝트가 커지면 다음처럼 확장할 수 있습니다.

```
Router
→ Service
→ Repository
→ Database
```

처음부터 너무 많은 파일을 만들면 오히려 흐름을 이해하기 어렵습니다.

따라서 처음에는 다음 구조를 추천합니다.

```
backend/
└── app/
    ├── main.py
    ├── database.py
    ├── models/
    ├── schemas/
    ├── services/
    └── routers/
```

---

8. 처음 배울 때 기억할 핵심

FastAPI 백엔드는 다음 네 부분만 먼저 기억하면 됩니다.

위치	역할
`routers/`	API 요청 받기
`services/`	실제 기능 처리
`models/`	DB 테이블 정의
`schemas/`	요청·응답 데이터 정의

전체 흐름은 다음과 같습니다.

```
Flutter
   ↓
Router
   ↓
Service
   ↓
Database
   ↓
응답 반환
```

그리고 User 기능은 다음처럼 배치하면 됩니다.

```
app/
├── models/
│   └── user.py
├── schemas/
│   └── user.py
├── services/
│   └── user_service.py
└── routers/
    └── user_router.py
```

가장 중요한 구분은 다음입니다.

```
models/user.py
→ 데이터베이스의 User

schemas/user.py
→ 요청과 응답에서 사용하는 User
```

이 두 가지를 구분할 수 있으면 FastAPI 폴더 구조의 절반 이상을 이해한 것입니다.